



Desarrollo Web

FACULTAD CIENCIAS DE LA INGENIERÍA, CARRERA SOFTWARE -
PRESENCIAL

Unidad 1

Lenguaje del lado del cliente – Front End

Tema 4

JavaScript

MSc. Josué Guevara
rguevarar@unemi.edu.ec

Objetivo

- Conocer la estructura de trabajo cuando se combina el desarrollo entre HTML, CSS y JAVASCRIPT, además de las etiquetas base y elementos principales de diseño.

Introducción

Tema:

¿Qué significa “programar” el comportamiento de un sitio web?

Introducción

JavaScript es un lenguaje de programación esencial en el desarrollo web, proporcionando interactividad y dinamismo a las páginas. La configuración básica del entorno de desarrollo es crucial para empezar a escribir y ejecutar código JavaScript de manera eficiente.

HTML

Estructura y contenido

CSS

Diseño visual y estilos

JavaScript

Comportamiento e interactividad

¿Qué puede hacer JS en una interfaz?

JavaScript no es solo lógica abstracta. En el contexto del frontend, actúa directamente sobre lo que el usuario ve y con lo que interactúa.



Responder a eventos

Detecta clics, entradas de texto, envíos de formularios y cualquier acción del usuario en tiempo real.



Validar formularios

Comprueba los datos antes de enviarlos, mostrando errores o confirmaciones directamente en pantalla.



Mostrar u ocultar

Controla la visibilidad de menús, modales, tooltips y paneles según el estado de la aplicación.



Actualizar contenido

Modifica el texto, las imágenes o la estructura de la página sin recargarla por completo.

Tecnologías con JavaScript

Vanilla JS

Término coloquial que se refiere a JavaScript sin librerías, plugins, frameworks, etc. Se puede utilizar como lenguaje de scripting y ofrece una actualización directa de los elementos de la interfaz de usuario.

Node.js

Framework de JavaScript que se utiliza para trabajar con el backend de una aplicación o para crearla con JavaScript. Es un entorno de ejecución multiplataforma, de código abierto, basado en el motor V8 de Google

React

Biblioteca de JavaScript de código abierto que se utiliza para crear interfaces de usuario. Es ideal para web applications y mobile applications.

Configuración básica de JavaScript

Antes de escribir código, es fundamental entender **cómo y dónde** incluir JavaScript en un proyecto web. Existen tres formas, pero solo una se recomienda para proyectos reales.

✗ Evitar

Inline: JS mezclado en el HTML

```
<button onclick="alert('Hola')">
```

Dificulta el mantenimiento y viola la separación de responsabilidades.

Interno: dentro de `<script>` en el HTML

Válido para pruebas rápidas, pero no para producción.

☑ Recomendado

Externo: archivo separado `app.js`

```
<script src="app.js" defer></script>
```

- Situar al final del `<body>` o usar `defer`
- Mantiene HTML, CSS y JS en archivos independientes
- Facilita el trabajo en equipo y la depuración

i Usa `defer` para garantizar que el DOM esté cargado antes de ejecutar el script.

Buenas prácticas desde el inicio

El código de calidad no es solo código que funciona: es código que se entiende, se mantiene y escala. Adoptad estos hábitos desde el primer día.

Separación de responsabilidades

Un archivo para HTML, otro para CSS, otro para JS. Nunca mezcles estilos o scripts inline en producción.

Nombres descriptivos

Usa nombres claros: `btnEnviar`, `listaProductos`, `mostrarError()`. Evita `x`, `temp` o `cosa`.

console.log para depurar

Es tu mejor aliado en desarrollo. Úsalo para comprobar valores, eventos y flujo de ejecución antes de usar el debugger.

Comentarios útiles

Comenta el *por qué*, no el *qué*. El código debe ser autoexplicativo; los comentarios añaden contexto de decisiones.

¿Qué es el DOM?

El **Document Object Model** es la representación en memoria que el navegador crea a partir del HTML. JavaScript puede leer y modificar esta estructura en tiempo real, sin recargar la página.

- ❗ El DOM no es HTML. Es la versión viva y manipulable de tu documento, que el navegador mantiene actualizada.

Lee el DOM

Accede al contenido, atributos y estilos de cualquier elemento

Modifica el DOM

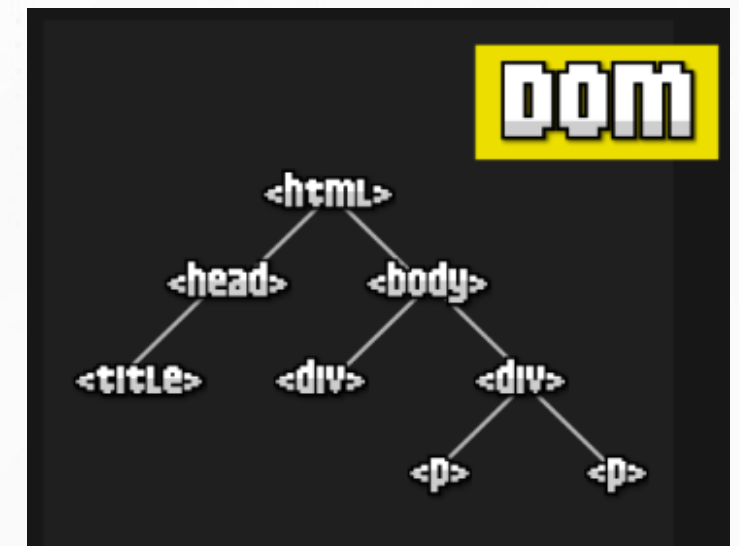
Cambia texto, clases CSS, visibilidad y estructura

Escucha el DOM

Detecta eventos y reacciona a las acciones del usuario

```
<html>
<head>
  <title>Title</title>
</head>
<body>
  <div></div>
  <div>
    <p>Párrafo 1</p>
    <p>Párrafo 2</p>
  </div>
</body>
</html>
```

HTML



Selección y Manipulación del DOM

Para actuar sobre un elemento HTML, primero debemos seleccionarlo. Los métodos modernos de JS hacen esto de forma muy similar a los selectores CSS que ya conocéis.

Seleccionar elementos

```
// Un solo elemento (como en CSS)
const titulo = document.querySelector('#titulo');
const btn = document.querySelector('.btn-primary');

// Varios elementos
const items = document.querySelectorAll('.producto');
```

Manipular contenido

```
titulo.textContent = 'Nuevo título';
titulo.innerHTML = '<b>Negrita</b>';
```

Manipular clases CSS

```
// Añadir clase
elemento.classList.add('activo');

// Eliminar clase
elemento.classList.remove('oculto');

// Alternar (toggle)
elemento.classList.toggle('visible');
```

✓ Con `classList.toggle` podéis mostrar/ocultar menús, modales y acordeones con una sola línea.

Seleccionar elementos del DOM

Existen una serie de métodos que nos permitirán buscar en el DOM de la página y encontrar dichos elementos.

Métodos de búsqueda	Descripción	Si no lo encuentra...
ELEMENT <code>.getElementById(id)</code>	Busca el elemento HTML por su <code>id</code> .	Devuelve <code>NULL</code> .
ARRAY <code>.getElementsByClassName(class)</code>	Busca elementos con la clase <code>class</code> .	Devuelve <code>[]</code> .
ARRAY <code>.getElementsByName(value)</code>	Busca elementos con el atributo <code>name</code> a <code>value</code> .	Devuelve <code>[]</code> .
ARRAY <code>.getElementsByTagName(tag)</code>	Busca etiquetas HTML <code>tag</code> .	Devuelve <code>[]</code> .

Ejemplos de selectores tradicionales

```
const page = document.getElementById("page"); // <div id="page"></div>
```

```
const items = document.getElementsByClassName("item"); // [div, div, div]
```

```
console.log(items[0]); // Primer item encontrado: <div class="item"></div>  
console.log(items.length); // 3
```

```
// Obtiene todos los elementos con atributo name="nickname"
```

```
const nicknames = document.getElementsByName("nickname"); // [input]
```

```
// Obtiene todos los elementos <div> de la página
```

```
const divs = document.getElementsByTagName("div"); // [div, div, div]
```

Ejemplos de selectores modernos

```
const page = document.querySelector("#page");           // <div id="page"></div>  
const info = document.querySelector(".main .info");     // <div class="info"></div>
```

```
// Obtiene todos los elementos con clase "info"  
const infos = document.querySelectorAll(".info");  
  
// Obtiene todos los elementos con atributo name="nickname"  
const nicknames = document.querySelectorAll('[name="nickname"]');  
  
// Obtiene todos los elementos <div> de la página HTML  
const divs = document.querySelectorAll("div");
```

Eventos: La Interactividad Real

Un **evento** es cualquier acción que ocurre en el navegador: el usuario hace clic, escribe, envía un formulario, mueve el ratón. JavaScript escucha esos eventos y responde con código.

1

El usuario actúa

Hace clic, escribe, envía...

2

El evento se dispara

click, input, submit

3

JS ejecuta la función

Modifica la UI en respuesta

```
const boton = document.querySelector('#btnEnviar');

boton.addEventListener('click', function() {
  document.querySelector('#mensaje').textContent = '¡Formulario enviado!';
  document.querySelector('#mensaje').classList.add('visible');
});
```

addEventListener y preventDefault

El método `addEventListener` es la forma moderna y recomendada de gestionar eventos. Permite añadir múltiples manejadores a un mismo elemento sin sobrescribir los anteriores.

Tipos de eventos clave

click

Botones, enlaces, tarjetas

input

Campos de texto en tiempo real

submit

Envío de formularios

Formulario con preventDefault

```
const form = document.querySelector('#miForm');

form.addEventListener('submit', function(e) {
  e.preventDefault(); // Evita recargar la página

  const nombre = document.querySelector('#nombre').value;

  if (nombre === '') {
    mostrarError('El nombre es obligatorio');
  } else {
    procesarFormulario(nombre);
  }
});
```

⚠ `preventDefault()` es esencial en formularios: sin él, el navegador recarga la página y pierdes todos los datos del usuario.

Ejemplo Práctico: Validación de Formulario

Este es el patrón más habitual en cualquier interfaz web. Un formulario de contacto o registro que valida los datos antes de permitir el envío, mostrando mensajes de error claros al usuario.

HTML (estructura)

```
<form id="formRegistro">
  <input type="text" id="email"
    placeholder="Tu email"/>
  <span id="errorEmail"
    class="error oculto"></span>
  <button type="submit">
    Registrarse
  </button>
</form>
```

JavaScript (comportamiento)

```
formRegistro.addEventListener('submit', (e) => {
  e.preventDefault();
  const email = document.querySelector('#email');
  const error = document.querySelector('#errorEmail');

  if (!email.value.includes('@')) {
    error.textContent = 'Email no válido';
    error.classList.remove('oculto');
    email.classList.add('campo-error');
  } else {
    // Continuar con el registro
    enviarDatos(email.value);
  }
});
```

Manipulación Dinámica de la Interfaz

Una de las capacidades más potentes de JavaScript es **crear y eliminar elementos HTML en tiempo real**, sin tocar el código fuente. Esto permite renderizar listas, tarjetas de productos, mensajes y cualquier contenido generado desde datos.

1

createElement

Crea un nuevo nodo HTML en memoria

2

Configurar

Añade texto, clases y atributos al elemento

3

appendChild

Inserta el elemento en el DOM visible

4

remove()

Elimina el elemento cuando ya no se necesita



Render Dinámico: Lista de Productos

Este patrón es la base de cualquier catálogo, carrito o feed de contenido. Los datos viven en un array de objetos y JavaScript genera el HTML automáticamente, manteniendo el código limpio y escalable.

Los datos (array de objetos)

```
const productos = [  
  { id: 1, nombre: 'Camiseta', precio: 25 },  
  { id: 2, nombre: 'Sudadera', precio: 49 },  
  { id: 3, nombre: 'Zapatillas', precio: 89 }  
];
```

El contenedor HTML

```
<ul id="listaProductos"></ul>
```

JavaScript renderiza la lista

```
const lista = document.querySelector('#listaProductos');  
  
productos.forEach(producto => {  
  const item = document.createElement('li');  
  item.classList.add('producto-item');  
  item.innerHTML = `  
    <span>${producto.nombre}</span>  
    <span>${producto.precio}€</span>  
    <button data-id="${producto.id}">  
      Añadir al carrito  
    </button>  
  `;  
  lista.appendChild(item);  
});
```

Librerías y Herramientas Modernas

JavaScript puro (Vanilla JS) es la base imprescindible. Una vez que lo dominéis, las librerías os permitirán construir interfaces más complejas con mayor eficiencia. Conocerlas es parte del perfil profesional moderno.



React

La librería más demandada. Construye interfaces como **componentes reutilizables**. Ideal para SPAs y apps complejas.



Vue.js

Alternativa progresiva y más sencilla de aprender. Perfecta para añadir interactividad a proyectos existentes.



Tailwind CSS

Framework CSS de clases utilitarias. Se integra perfectamente con React y Vue para construir UIs rápidamente.



Bootstrap

El framework más conocido para prototipar interfaces responsive con componentes listos para usar vía CDN.

JS Puro vs. Librerías: ¿Cuándo usar cada uno?

Esta es una pregunta que os haréis constantemente en proyectos reales. La respuesta depende del alcance, el equipo y los objetivos del proyecto.

☑ Usar JavaScript puro cuando...

- El proyecto es pequeño o mediano
- Necesitáis control total del código
- Estáis aprendiendo fundamentos
- No queréis dependencias externas
- La lógica de UI es simple y puntual

⚡ Usar una librería cuando...

- La interfaz tiene muchos estados y componentes
- Trabajáis en equipo con estándar común
- El proyecto es una SPA o app compleja
- Necesitáis productividad y ecosistema maduro
- El cliente o empresa ya usa ese stack

📄 Regla práctica: **primero entiende la base**, luego usa la herramienta. Una librería no sustituye entender el DOM.

Errores Comunes que Debéis Evitar

Conocer los errores más frecuentes os ahorrará horas de depuración. Estos son los patrones problemáticos que aparecen en casi todos los proyectos de principiantes.



JS sin conectar al HTML

Escribís el script pero el selector no existe o el archivo no está enlazado.
Comprobad siempre el `id` o `class` y el `defer`.



Eventos mal gestionados

Asignar eventos con `onclick=""` inline o no usar `addEventListener`. El código se vuelve inmantenible y difícil de depurar.



No validar formularios

Confiar en que el usuario introducirá datos correctos. Siempre validar: campos vacíos, formatos, longitudes mínimas.



Código desordenado

Todo en un único fichero sin funciones, sin nombres claros.
Organizad el código en funciones con una sola responsabilidad.



Dependencia total de IA

Copiar y pegar sin entender. Si no sabéis lo que hace el código, no podréis corregirlo cuando falle en producción.

BIBLIOGRAFÍA

- Mastermarketing-valencia.com. (2019). ¿Cuándo nació Internet? Historia y evolución. Recuperado de <https://n9.cl/hlnxf>
- Mastermarketing-valencia.com. (2019). Infografía-La web Historia. Recuperado de <https://n9.cl/u7bct>
- Significado de Web. (2021). En Significados. Recuperado de <https://n9.cl/5gpv4>
- Schiaffarino, A. (2019). Modelo cliente servidor. Infranetworking. Recuperado de <https://n9.cl/bee5>
- ARQUITECTURA CLIENTE-SERVIDOR. (2021). Nota Tecnológica. Recuperado de <https://n9.cl/kc28a>
- Berners-Lee, T. (2000). Weaving the web: The original design and ultimate destiny of the World Wide Web. Harper Business.
- Tanenbaum, A. S., & van Steen, M. (2001). Distributed systems: Principles and paradigms (1st ed.). Pearson.
- Robbins, J. N. (2012). Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics (4th ed.). O'Reilly Media.

UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO